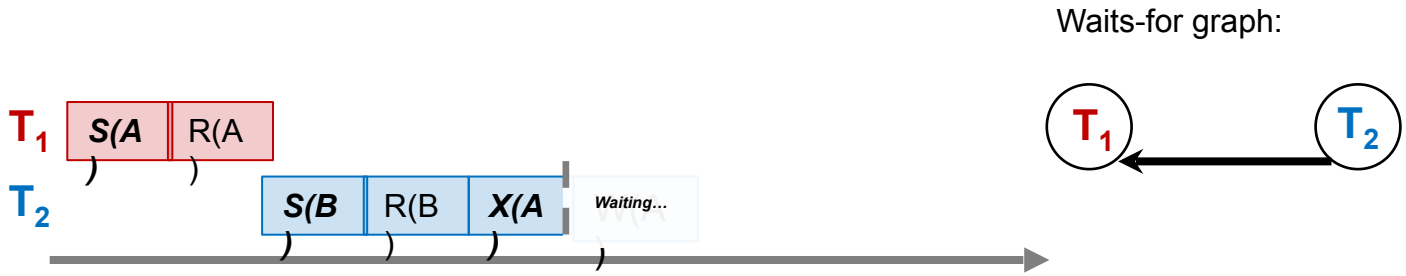
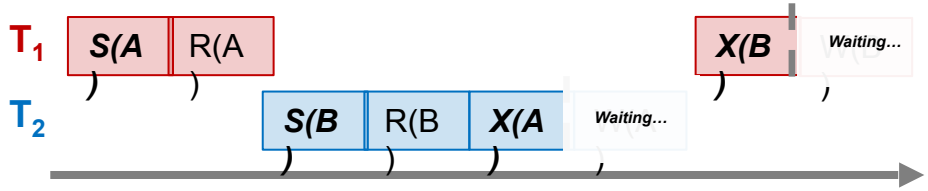
Deadlock Detection: Example



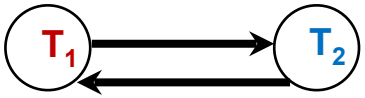
T_2 then requests an exclusive lock on A to write to it- **now T_2 is waiting on T_1…**

Waits-For graph: Track which Transactions are waiting
IMPORTANT: WAITS-FOR graph different than CONFLICT graph we learnt earlier !

Deadlock Detection: Example



Waits-for graph:



Cycle = DEADLOCK

Finally, T_1 requests an exclusive lock on B to write to it- **now T_1 is waiting on T_2… DEADLOCK!**

Deadlocks

Deadlock: Cycle of transactions waiting for locks to be released by each other.

Two ways of dealing with deadlocks:

- Deadlock prevention
- Deadlock detection

Deadlock Prevention: C2PL

Conservative 2 Phase Locking (C2PL)

- Obtains all locks before the transaction begins
- If cannot obtain ALL locks, release and try again

- Ensures that no deadlocks occurs
- BUT: can degrade performance

Deadlock Prevention: Wait-Die / Wound-Wait

- Transactions assigned an ID (e.g., based on their start time)
  - T1, T2, …
  - ID determines priority (priority of T1 > T2 > T3, …)
  - If Tx wants a lock that Ty holds, we have two deadlock avoidance algorithms

- Scheme 1: Wait-Die (non-preemptive):
  - If Tx has higher priority, Tx waits for Ty, else Tx aborts
- Scheme 2: Wound-Wait (preemptive):
  - If Tx has higher priority, Ty aborts, else Tx waits

- In both cases aborted transactions start again some random later with the same time stamp (priority)
- What is the shape of the wait graph under both of these schemes?
- What are the pros and cons of each?

Alternative approach: deadlock detection

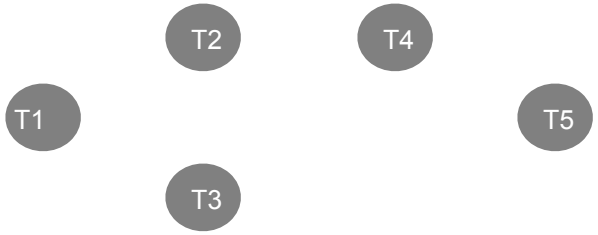
Create the **waits-for graph**:

- Nodes are transactions
- There is an edge from $T_i \rightarrow T_j$ if T_i is *waiting for T_j to release a lock*

Periodically check for (***and break***) cycles in the waits-for graph

Example with 5 Transactions (2PL)

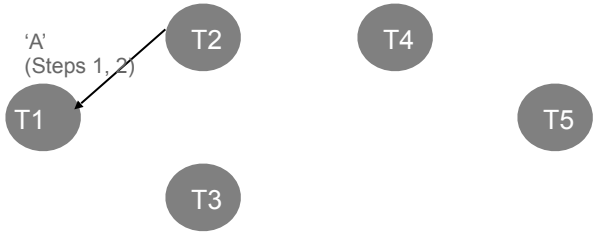
Operation Arrival	w1(A)	r2(A)	w1(B)	w3(C)	r2(C)	r4(B)	w2(D)	w4(E)	r5(D)	w5(E)
Execute with 2PL	T1	T2		T3		T4			T5	



Waits- For Graph

Example with 5 Transactions (2PL)

Operation Arrival	w1(A)	r2(A)	w1(B)	w3(C)	r2(C)	r4(B)	w2(D)	w4(E)	r5(D)	w5(E)
Execute with 2PL	T1	T2		T3		T4			T5	
Step 0	X (A) w1(A)									
Step 1		Req S(A)								
Step 2	X (B) w1(B) Unl B, A									
Step 3		Get S(A) r2(A)								
Step 4			X (C) w3(C) Unl C							
Step 5		S(C) r2(C)								
Step 6				S(B) r4(B)						
Step 7		X(D) w2(D) Unl A, C, D								
Step 8				X(E) w4(E) Unl B,E						
Step 9					S(D) r5(D)					
Step 10					X (E) w5(E) Unl D, E					



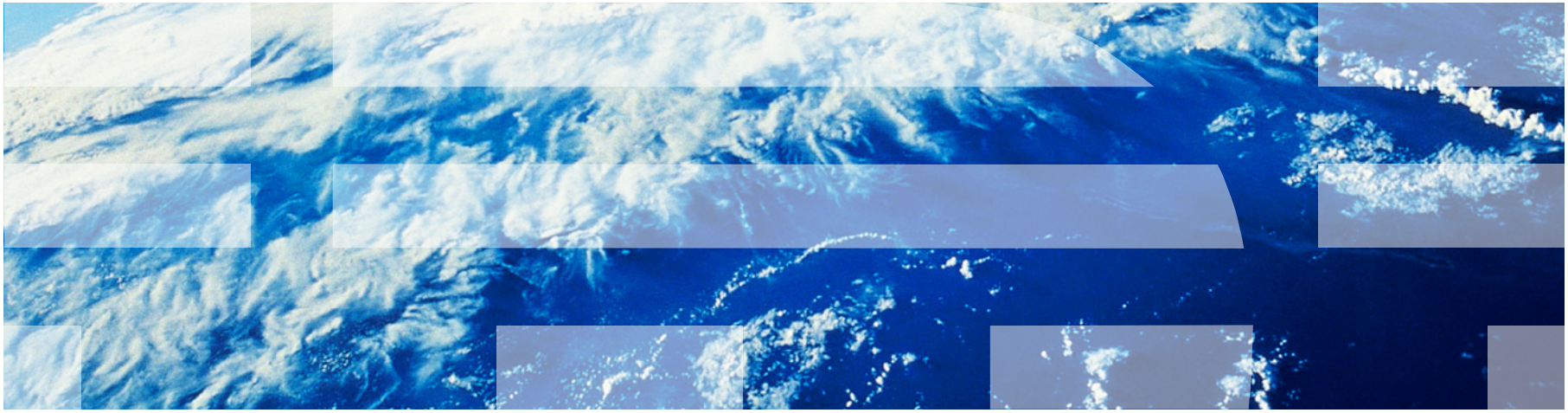
Waits- For Graph

Summary

Locking allows only conflict serializable schedules

- If the schedule completes… (it may deadlock!)
- → No guarantee that the schedule will finish!

Other Isolation Levels Beyond Serializability



Real-world Isolation Guarantees

- Serializability is unfortunately rare!
- Some databases claim to be serializable, but actually are not!

- **Read committed**: very common in SQL OLTP
  - Any data that was read was committed
  - Disallows:
    - Dirty reads
  - Allows:
    - Lost updates
    - Unrepeatable reads
    - Phantom reads

- **Repeatable reads**: stronger than read committed, by still allows lost updates and phantom reads

Strict serializability, the gold standard

- Limitations of serializability:
  - No constraints on equivalent order of transactions
  - No relationship between final serial order and when the transactions were submitted
  - Two databases given identical set of transactions in the same order at the same time may produce very different results

- Strict serializability is even stronger than conflict serializability
- Definition: If transaction Y starts after transaction X commits (X and Y are not concurrent)
  1. Final state is equivalent to serial order
  2. X must have been executed before Y in that serial order

2PL produces **strictly serializable** schedules

Why?